

1Z0-1157-26: Agentic AI Foundations Associate

Comprehensive Practice Exam & Answer Key • Modules 1 - 5 • 30 Scenario Questions

Syllabus Coverage & Exam Information

This practice exam features **30 highly realistic, scenario-based multiple-choice questions** designed to align perfectly with the Oracle University Agentic AI Foundations curriculum. Use this companion guide to test your readiness across all five key modules before sitting for the official proctored certification exam (1Z0-1157-26).

HIGH-YIELD EXAM ALERT: Remember these absolute rules for the exam: (1) The LLM's reasoning engine remains completely **invariant** under MCP or any SDK. The model only evaluates schemas and emits intents—it never runs tool code directly. (2) In local stdio subprocess environments (like local MCP or uvx), the client application script **never speaks HTTPS directly to cloud backends**; network connections are handled solely by the isolated server process. (3) Over 99% of total agent latency stems from **LLM API network round-trips**, not local code.

Part I: Practice Exam Questions

Question 1: What is the core execution loop difference between a standalone LLM and an LLM-based AI Agent?

- A) Standalone LLMs are stateful, whereas AI Agents are entirely stateless and cannot persist conversational context.
- B) Standalone LLMs operate on a single-pass inference response, whereas AI Agents execute iteratively inside a cyclical loop (Perceive → Reason → Act → Observe).
- C) Standalone LLMs require real-time tools for basic text generation, whereas AI Agents are restricted to pre-trained static knowledge.
- D) AI Agents run on local client-side servers, whereas standalone LLMs are strictly cloud-hosted.

Question 2: In the 'Two-tier LLM architecture pattern' for AI Agents, what are the respective roles of the models?

- A) The primary model handles hard-coded safety guardrails, while the secondary model manages user registration.
- B) A cheap, low-latency model is used for routine routing and tool dispatching, while a high-capacity model is reserved for advanced reasoning tasks.
- C) An open-source model performs vector embedding generation, while a proprietary model indexes the database.
- D) Both tiers run concurrently to average out token cost and speed up API response times.

Question 3: During an agent's tool execution cycle, why is it considered a critical security practice to keep tool code execution outside of the LLM?

- A) The LLM lacks the memory capacity to cache tool results.
- B) Running code inside the LLM increases token costs exponentially.
- C) The LLM never executes tool code directly; it only outputs structured text (e.g., tool-call JSON) which is validated and run by the backend application environment.
- D) Standardizing execution inside the LLM violates the Model Context Protocol (MCP) spec.

Question 4: How does an agentic framework (such as LangChain or SmolAgents) communicate a registered function's capabilities to the LLM?

- A) It uploads the compiled binary directly to the LLM's system prompt.
- B) It converts the Python function's docstring and type hints into a structured JSON schema that the LLM reads.
- C) It translates the Python code into SQL queries for execution.
- D) It queries an external API catalog whenever the user submits a prompt.

Question 5: Which of the following describes a key limitation of standalone Chain-of-Thought (CoT) prompting in agentic tasks?

- A) It is incapable of generating structured markdown or JSON outputs.
- B) It relies entirely on internal knowledge, cannot run tools to look up real-time facts, and cannot self-correct against external reality.
- C) It requires model fine-tuning and cannot be triggered via simple system prompts.
- D) It can only handle mathematical equations and is useless for conversational queries.

Question 6: Why does the ReAct (Reasoning + Acting) framework significantly reduce hallucinations compared to standard prompting?

- A) It fine-tunes the model on specific domain-focused databases.
- B) It completely bypasses intermediate reasoning steps to output answers immediately.
- C) It interleaves reasoning traces with tool Actions and grounds subsequent reasoning steps on real-world Observations.
- D) It forces the model to run in a localized, offline environment with zero external access.

Question 7: In the orchestration layer (the agent's nervous system), which module is specifically responsible for managing short-term scratchpads and long-term persistent history?

- A) Safety & Guardrails
- B) State Machine
- C) Memory Management
- D) Tool Routing

Question 8: An attacker plants malicious invisible instructions in a public web document. When an agent retrieves this document during a RAG lookup, the instructions hijack the session to steal sensitive data. What type of vulnerability is this?

- A) Runaway Execution
- B) Direct Prompt Injection
- C) Memory Poisoning
- D) Indirect Prompt Injection

Question 9: An attacker poisons an agent's long-term storage, altering how the agent behaves in all future sessions. Which security risk does this exploit represent?

- A) Indirect Prompt Injection
- B) Memory Poisoning
- C) Runaway Execution
- D) Direct API Exfiltration

Question 10: Under the defense-in-depth framework, which layer of the guardrails architecture is responsible for intercepting and screening toxic, unauthorized, or private data (PII) before it is returned to the user?

- A) Input Validation
- B) Output Filtering
- C) Tool Boundaries
- D) Observability

Question 11: A development team is building a system where multiple specialist agents (e.g., a researcher, an auditor, and a copywriter) must collaborate as a cohesive unit. Which framework is specifically designed for multi-agent role-based workflows?

- A) OpenAI Agents SDK
- B) Hugging Face SmolAgents
- C) CrewAI
- D) Standalone Chain-of-Thought

Question 12: What is the primary purpose of the Model Context Protocol (MCP) in modern agentic architectures?

- A) To bypass the application backend and let the LLM execute system shell commands directly.
- B) To provide a standardized, open-source protocol for securely connecting LLMs with the data, prompts, and tools they need.
- C) To fine-tune the weights of low-capacity routing models.
- D) To compress JSON schemas to reduce total input token overhead.

Question 13: An engineer wants to design a system where a primary triage agent delegates complex sub-tasks to specialized sub-agents. According to the OpenAI Agent Stack architecture, which layer natively supports this multi-agent delegation pattern and handles automated ReAct loop orchestration out of the box?

- A) Layer 1: OpenAI Models
- B) Layer 2: Responses API
- C) Layer 3: Agents SDK
- D) Layer 4: Application Layer

Question 14: In the OpenAI Responses API, how does conversation state propagation work, and what is its primary benefit over the legacy Chat Completions API?

- A) It serializes and resends the entire message array on each request, maximizing client-side security.
- B) It uses the previous_response_id pointer, storing conversation history on the client-side for immediate offline debugging.
- C) It passes a previous_response_id to point to the prior response, prompting OpenAI servers to resolve context server-side and drastically reducing network bandwidth and input tokens.
- D) It automatically deploys a local Redis vector database instance to cache and index multi-turn interactions.

Question 15: When registering a custom Python function using the @function_tool decorator in the OpenAI Agents SDK, what are the respective roles of Python type hints and docstrings in compiling the model's toolbox?

- A) Type hints determine the execution permissions of the sandbox container, while docstrings specify the API key required for OAuth2.
- B) Type hints are compiled into the JSON input schema to validate arguments, while docstrings serve as the semantic description the LLM evaluates to decide when to call the tool.
- C) Type hints determine the return speed of the function, while docstrings are discarded during compilation to optimize token context.
- D) Type hints register the function with remote MCP servers, while docstrings are used by LangSmith to trace performance bottlenecks.

Question 16: An enterprise retail app uses a frontline triage agent to route customer queries. Once a query is classified, the triage agent delegates the issue to a Billing Specialist, who then takes over the dialogue and issues the final invoice refund directly to the user. Which multi-agent pattern does this represent, and what is the context visibility rule for the recipient specialist?

- A) Manager Pattern; the billing specialist has zero visibility into the conversation prior to the delegation.
- B) Handoffs Pattern; the billing specialist receives the full conversation history to maintain context.
- C) Manager Pattern; the billing specialist receives only the latest user message to protect customer PII.
- D) Handoffs Pattern; the billing specialist must re-prompt the user for basic details as control flow cannot pass state.

Question 17: In the OpenAI Agents SDK safety architecture, an input guardrail decorated with `@input_guardrail` is attached to a main router agent. What is the chronological execution order of this guardrail, and what happens when its evaluation conditions return `tripwire_triggered = True`?

- A) It runs in parallel with tool execution; triggering a tripwire causes the agent to discard the tool observation but proceed with text generation.
- B) It runs after the LLM decides to call a tool; triggering a tripwire wipes the agent's short-term session memory.
- C) It runs before the main agent or LLM processes the user prompt; triggering a tripwire fires an immediate exception that halts execution, blocking downstream tool calls and saving token costs.
- D) It runs only when the model has completed its ReAct loop; triggering a tripwire forces the client to retry the request up to 3 times.

Question 18: A developer uses the OpenAI Platform dashboard to review a trace of a multi-agent workflow that took 12 seconds to complete. The trace shows that the support checker, triage routing, and specialist agent each spent 2 to 5 seconds inside `POST /v1/responses`, while the custom local database tool ran in 3 milliseconds. What optimization directive should the engineer follow to reduce end-to-end wall-clock latency?

- A) Rewrite the database tool in a faster language like C++ to reduce execution latency.
- B) Minimize the number of sequential LLM API round-trips by merging steps, tuning prompts, or using smaller non-reasoning models for triage.
- C) Move tool execution from the local child subprocess to a remote streamable HTTP server.
- D) Disable the Conversations API to prevent state tracking on OpenAI servers.

Question 19: Which of the following correctly pairs an OpenAI Agents SDK tool category with its runtime execution location?

- A) Hosted Tools (like `web_search` or `code_interpreter`) execute locally inside the developer's workstation terminal subprocess.
- B) Function Tools (with `@function_tool`) execute on OpenAI-managed remote servers, requiring secure API keys to be uploaded.
- C) Hosted Tools (like `file_search` or `web_search`) run natively on OpenAI's managed infrastructure with zero local execution plumbing.
- D) Agents as Tool run inside isolated local Docker containers managed directly by the OCI registry (OCIR).

Question 20: In the centralized Manager Pattern, how does the manager agent coordinate specialized worker agents under the hood?

- A) It uses the `Handoffs` array to relinquish control, letting specialists take over user interactions directly.
- B) It packages specialized worker agents into tools using `agent.as_tool` and retains primary execution control, synthesizing their responses into a unified final output.
- C) It routes all traffic through standard input/output pipes to bypass LLM reasoning.
- D) It spawns worker agents as local child subprocesses via `uvx` and communicates with them over JSON-RPC 2.0.

Question 21: How is the OCI Enterprise AI Platform organized across its three main layers, and which layer is specifically responsible for managed tools, MCP integration, and Natural Language to SQL?

- A) Brain Layer (Governance), Action Layer (Models), Trust Layer (Agents); NL-to-SQL is managed by the Models layer.
- B) Brain Layer (Models), Action Layer (Agents), Trust Layer (Governance); managed tools, MCP, and NL-to-SQL are coordinated by the OCI Enterprise AI Agents Service (Action Layer).
- C) Brain Layer (Models), Action Layer (Governance), Trust Layer (Agents); the Vector Stores API is managed by the Trust Layer.
- D) Brain Layer (Agents), Action Layer (Models), Trust Layer (Governance); NL-to-SQL is managed by the zero-trust packet routing service.

Question 22: In a Retrieval-Augmented Generation (RAG) system deployed on the OCI Enterprise AI Platform, what are the respective roles and metrics of embedding models and reranking models?

- A) Embedding models determine precision by scoring document relevance, while rerank models determine recall by mapping text to numerical vectors.
- B) Embedding models determine recall by mapping user queries and documents into a geometric vector space for fast broad retrieval; reranking models determine precision by re-scoring candidates against the query using a more expensive, accurate relevance model.
- C) Embedding models convert text into SQL statements, while rerank models execute the query in Oracle Database.
- D) Embedding models run only on-demand, while rerank models require Dedicated AI Clusters.

Question 23: An enterprise is deploying a fine-tuned, proprietary model for an automated financial analyst agent. Which serving mode must they select on OCI, and what is a primary model lifecycle benefit of this mode?

- A) On-Demand; they are billed only per call, and Oracle handles all model retirement automatically without service disruption.
- B) Dedicated AI Clusters (DAC); they have single-tenant GPU physical isolation, and existing clusters continue running retired models without interruption even if Oracle deprecates the base model.
- C) On-Demand; they get physical single-tenant GPU isolation and SLA latency guarantees.
- D) Dedicated AI Clusters (DAC); they pay per token with zero upfront cluster hours commitment.

Question 24: Under the OCI Generative AI Security framework, how are the three layers of the defense-in-depth model mapped to secure model inference?

- A) Network (IAM policies), Identity (Private Endpoints), Behavioral (ZTPR Zero Trust Packet Routing).
- B) Network (Private Endpoints & ZTPR), Identity (OCI IAM policies), Behavioral (Runtime Guardrails for content moderation and PII detection).
- C) Network (Runtime Guardrails), Identity (Private Endpoints), Behavioral (OCI IAM compartment scoping).
- D) Network (ZTPR), Identity (Runtime Guardrails), Behavioral (Compartment policies).

Question 25: A startup has an existing multi-agent application built on the OpenAI Agents SDK and wants to migrate it to OCI Enterprise AI Agents Service. Which code modifications are required to achieve this migration?

- A) Completely rewrite the application in Java as the OCI Responses API does not support Python or OpenAI SDK syntax.
- B) Re-point the base URL to OCI endpoints, update the authentication headers to use OCI IAM credentials, and configure requests to target OCI enterprise AI models.
- C) De-register all `@function_tool` decorators and define tools natively in the OCI web console.
- D) Deploy an external MCP server to translate OpenAI SDK commands to OCI REST payloads.

Question 26: To deploy a custom multi-agent application as an OCI Hosted Application, what is the required operational packaging and deployment pipeline?

- A) Write the code in the OCI Console, register tools via PyPI, and launch via `uvx`.
- B) Package the agent application into a container image, upload the image to OCI Registry (OCIR), and deploy it as a managed application that exposes a dedicated HTTPS endpoint.
- C) Convert the agent code into SQL PL/SQL procedures and load them into Oracle Autonomous Database.
- D) Push the raw `.py` scripts directly to the OCI Object Storage bucket and run them via a serverless function.

Question 27: The OCI Enterprise AI Agents Service provides three native state management tiers. Which of the following correctly describes their functional roles?

- A) Short-Term Memory (Files API), Long-Term Memory (Vector Stores API), Context Compaction (Containers API).
- B) Short-Term Memory (Conversations API session dialogue history), Long-Term Memory (cross-session user preferences and profiles), Context Compaction (automatic conversational summarization to fit token limits).
- C) Short-Term Memory (local RAM), Long-Term Memory (Object Storage), Context Compaction (manual text truncation).
- D) Short-Term Memory (JSON-RPC), Long-Term Memory (stdio pipes), Context Compaction (HTTPS).

Question 28: An OCI Usage MCP Server is published on PyPI and executed locally via `uvx` over the stdio transport. When the client application invokes the `get_summarized_usage` tool, how do network isolation and security boundaries operate between the client, MCP server, and OCI cloud?

- A) The client application script opens a direct HTTPS socket to the public OCI REST API to execute the query.
- B) The client and the MCP server communicate locally over standard input/output (stdio) pipes on the host machine with zero network traffic; only the isolated MCP server process initiates outbound HTTPS calls to the OCI Cloud API.
- C) The client speaks to the MCP server over Streamable HTTP SSE, requiring port-forwarding and network gateways on the local machine.
- D) The LLM intercepts the stdio stream and executes the OCI REST API logic directly within its model weights.

Question 29: What is the primary safety function of the managed Code Interpreter Tool in OCI Enterprise AI Agents Service?

- A) To convert natural language prompts into secure SQL statements to query databases.
- B) To run model-generated Python code inside an isolated, secure compute sandbox with strict CPU, memory, and runtime limits to prevent host system compromise.
- C) To compile Python code into container images for OCI Registry deployment.
- D) To audit LLM output against prompt injection templates.

Question 30: How does the OCI Agents Service native integration with Oracle AI Database enable access to federated enterprise structured data?

- A) By automatically generating and executing raw Python code to edit database tables locally.
- B) By leveraging Natural Language to SQL (NL-to-SQL) capabilities to translate natural language user questions into secure SQL statements and query structured data residing in Oracle Autonomous Databases.
- C) By forcing the model to embed the entire SQL database into high-dimensional vector embeddings.
- D) By migrating all structured tables into passive flat text files registered in the OCI Files API.

Part II: Comprehensive Answer Key & Explanations

Explanation for Question 1:

CORRECT ANSWER: B

Correct Option details: Standalone LLMs operate on a single-pass inference response, whereas AI Agents execute iteratively inside a cyclical loop (Perceive → Reason → Act → Observe).

A standalone LLM is a single-pass inference engine—it receives a prompt and outputs a response without any dynamic iteration. An AI Agent, however, operates inside a closed loop (Perceive → Reason → Act → Observe) that runs iteratively until a final goal is reached or a guardrail triggers a stop.

Explanation for Question 2:

CORRECT ANSWER: B

Correct Option details: A cheap, low-latency model is used for routine routing and tool dispatching, while a high-capacity model is reserved for advanced reasoning tasks.

The two-tier LLM architecture pattern is an optimization strategy to balance cost and latency. A small, fast, cheap model is utilized at the front-end to classify user intent, handle basic routing, and manage simple tool routing. When the agent detects that deep logical thinking or complex reasoning is required, it routes the sub-task to a heavier, more expensive reasoning-focused model.

Explanation for Question 3:

CORRECT ANSWER: C

Correct Option details: The LLM never executes tool code directly; it only outputs structured text (e.g., tool-call JSON) which is validated and run by the backend application environment.

Letting the LLM execute tool code directly is a massive security hazard and technically impossible for basic models. In safe architectures, the LLM is restricted to a text-in, text-out interface. It emits structured text (typically a JSON schema) expressing its intent to call a tool. The hosting application code intercepts this text, validates the input parameters, and executes the actual code inside a secure boundary safely separated from the model.

Explanation for Question 4:

CORRECT ANSWER: B

Correct Option details: It converts the Python function's docstring and type hints into a structured JSON schema that the LLM reads.

Agentic frameworks leverage Python metadata to automate tool discovery. By parsing a function's docstring (which describes what the tool does) and its parameter type hints, the framework automatically compiles a JSON schema. This schema is appended to the system prompt of the LLM, enabling the model to match user queries to the specific tool's metadata.

Explanation for Question 5:

CORRECT ANSWER: B

Correct Option details: It relies entirely on internal knowledge, cannot run tools to look up real-time facts, and cannot self-correct against external reality.

Standalone Chain-of-Thought (CoT) prompting enables step-by-step thinking, but it operates entirely within the model's static, internal parameter weights. If those weights contain incorrect information, the step-by-step reasoning will simply be a logical sequence leading to a confidently wrong answer. It lacks the mechanism to invoke external tools, fetch live data, or modify its plan based on external observations.

Explanation for Question 6:

CORRECT ANSWER: C

Correct Option details: It interleaves reasoning traces with tool Actions and grounds subsequent reasoning steps on real-world Observations.

The ReAct (Reasoning + Acting) pattern reduces hallucinations because it grounds its reasoning steps in real-world reality. Instead of relying solely on internal weights (like CoT), it performs an Action (calls a tool), receives an actual Observation (tool output), and then uses that Observation to form its next Thought. This interleaving cycle prevents the model from fabricating false links.

Explanation for Question 7:

CORRECT ANSWER: C

Correct Option details: Memory Management

Memory Management is the orchestration loop component that handles an agent's storage. It acts as a short-term scratchpad for the active execution run (tracking intermediate thoughts and observations) and connects to long-term data stores (e.g., semantic vectors, conversation history, databases) to maintain context across multi-turn sessions.

Explanation for Question 8:

CORRECT ANSWER: D

Correct Option details: Indirect Prompt Injection

This is a classic Indirect Prompt Injection. Unlike direct injection, where the user types malicious instructions directly into the chat bar, indirect injection occurs when the agent autonomously reads external data (such as a poisoned text file, email, or web page) containing invisible or disguised commands that hijack the LLM's active orchestration loop.

Explanation for Question 9:

CORRECT ANSWER: B

Correct Option details: Memory Poisoning

Memory Poisoning occurs when an attacker writes malicious instructions into an agent's persistent long-term memory. Since the memory is retrieved across subsequent user sessions, the exploit persists silently and can affect future decisions, leading to ongoing vulnerabilities such as secret data exfiltration or state corruption.

Explanation for Question 10:

CORRECT ANSWER: B

Correct Option details: Output Filtering

Under a layered defense-in-depth guardrail framework, Output Filtering is the final line of defense. It acts as a filter that screens all generated responses for toxic content, PII leaks, API keys, or compliance violations before the text is displayed to the user.

Explanation for Question 11:

CORRECT ANSWER: C

Correct Option details: CrewAI

CrewAI is specifically built around multi-agent team mechanics. It allows developers to define individual, role-based agents with custom tools and personas (e.g., Researcher, Auditor) and choreograph their execution as a collaborative 'crew' with structured hand-offs.

Explanation for Question 12:

CORRECT ANSWER: B

Correct Option details: To provide a standardized, open-source protocol for securely connecting LLMs with the data, prompts, and tools they need.

The Model Context Protocol (MCP) is an open-standard protocol designed to simplify how LLMs connect with external data sources, prompts, and tools. Instead of custom, fragmented integration layers, MCP provides a secure, unified protocol for developers to feed rich context into LLMs.

Explanation for Question 13:

CORRECT ANSWER: C

Correct Option details: Layer 3: Agents SDK

In the OpenAI Agent Stack, the Agents SDK (Layer 3) handles complex multi-agent workflows, handoffs, and orchestration loop automation out of the box. Layer 2 (Responses API) supports tool calling and memory but requires the developer to manually write the loop logic and context forwarding code.

Explanation for Question 14:

CORRECT ANSWER: C

Correct Option details: It passes a `previous_response_id` to point to the prior response, prompting OpenAI servers to resolve context server-side and drastically reducing network bandwidth and input tokens.

The OpenAI Responses API maintains stateful conversation chains on the server-side by linking requests via the `previous_response_id` pointer. This avoids client-side history serialization and redundant token re-transmissions over network hops, optimizing bandwidth and costs.

Explanation for Question 15:

CORRECT ANSWER: B

Correct Option details: Type hints are compiled into the JSON input schema to validate arguments, while docstrings serve as the semantic description the LLM evaluates to decide when to call the tool.

Under function calling, type hints (e.g., param: float) determine the data types and parameter bounds compiled into the JSON inputSchema. Docstrings are compiled into the natural language 'description' field, which is read by the LLM to perform semantic match evaluations of when to use the tool.

Explanation for Question 16:

CORRECT ANSWER: B

Correct Option details: Handoffs Pattern; the billing specialist receives the full conversation history to maintain context.

In the decentralized Handoffs pattern, control of the dialogue is transferred from the Triage Agent directly to a target Specialist Agent. The specialist takes complete control of the loop and interacts with the user directly, and natively receives the full dialogue history array to preserve user context.

Explanation for Question 17:

CORRECT ANSWER: C

Correct Option details: It runs before the main agent or LLM processes the user prompt; triggering a tripwire fires an immediate exception that halts execution, blocking downstream tool calls and saving token costs.

Input guardrails are evaluated at the perimeter, before the main agent router executes its reasoning cycle. When a tripwire triggers (tripwire_triggered: True), an immediate exception is raised to halt the thread before any downstream tool call or model completion runs, preventing token waste or unsafe action leaks.

Explanation for Question 18:

CORRECT ANSWER: B

Correct Option details: Minimize the number of sequential LLM API round-trips by merging steps, tuning prompts, or using smaller non-reasoning models for triage.

Network round-trips to the LLM API represent the dominant latency bottleneck in production systems (accounting for >99% of wall-clock time). Local logic, guardrail checkers, handoff pointer swaps, and code executions complete in single-digit milliseconds. Therefore, optimization must focus on reducing sequential LLM requests.

Explanation for Question 19:

CORRECT ANSWER: C

Correct Option details: Hosted Tools (like file_search or web_search) run natively on OpenAI's managed infrastructure with zero local execution plumbing.

Hosted Tools (like web_search, file_search, and code_interpreter) run natively on OpenAI's fully managed, remote cloud infrastructure. They require zero custom execution scripts, dependencies, or process containers on the developer's local workstation.

Explanation for Question 20:

CORRECT ANSWER: B

Correct Option details: It packages specialized worker agents into tools using agent.as_tool and retains primary execution control, synthesizing their responses into a unified final output.

In the centralized Manager Pattern, the manager agent does not relinquish control. Instead, specialized subordinate worker agents are registered as tools under the manager using the agent.as_tool utility, allowing the manager to delegate tasks, gather findings, and synthesize a single, unified final output.

Explanation for Question 21:

CORRECT ANSWER: B

Correct Option details: Brain Layer (Models), Action Layer (Agents), Trust Layer (Governance); managed tools, MCP, and NL-to-SQL are coordinated by the OCI Enterprise AI Agents Service (Action Layer).

OCI Enterprise AI Platform unifies enterprise intelligence across three layers: OCI Models (Brain), OCI Agents (Action), and OCI Governance (Trust). Custom tools, MCP connectivity, and natural language database capabilities are coordinates of the OCI Enterprise AI Agents Service (Action layer).

Explanation for Question 22:

CORRECT ANSWER: B

Correct Option details: Embedding models determine recall by mapping user queries and documents into a geometric vector space for fast broad retrieval; reranking models determine precision by re-scoring candidates against the query using a more expensive, accurate relevance model.

In OCI RAG, Embedding models perform fast vector math to maximize recall (broad document matching). Reranking models execute deep, more expensive semantic relevance comparisons on retrieved candidates to filter down to the top 5-10 high-relevance items, maximizing precision before text generation.

Explanation for Question 23:

CORRECT ANSWER: B

Correct Option details: Dedicated AI Clusters (DAC); they have single-tenant GPU physical isolation, and existing clusters continue running retired models without interruption even if Oracle deprecates the base model.

Dedicated AI Clusters (DAC) are required to import and host fine-tuned models, providing physical single-tenant GPU isolation and predictable latency. Crucially, when Oracle retires a base model, any existing active dedicated cluster for that model continues running without service interruption.

Explanation for Question 24:

CORRECT ANSWER: B

Correct Option details: Network (Private Endpoints & ZTPR), Identity (OCI IAM policies), Behavioral (Runtime Guardrails for content moderation and PII detection).

OCI secures model interactions through three defensive layers: Network Security (private endpoints and Zero Trust Packet Routing to keep traffic isolated), Identity controls (compartment-scoped OCI IAM policies), and Behavioral safety (runtime guardrails for content moderation, jailbreak blocks, and PII filters).

Explanation for Question 25:

CORRECT ANSWER: B

Correct Option details: Re-point the base URL to OCI endpoints, update the authentication headers to use OCI IAM credentials, and configure requests to target OCI enterprise AI models.

The OCI Responses API conforms to the open Responses API specification and is fully OpenAI-compatible. This allows migrating applications written on the OpenAI Agents SDK by merely re-pointing the base URL to OCI endpoints, updating authentication headers to use OCI IAM credentials, and selecting OCI enterprise AI models.

Explanation for Question 26:

CORRECT ANSWER: B

Correct Option details: Package the agent application into a container image, upload the image to OCI Registry (OCIR), and deploy it as a managed application that exposes a dedicated HTTPS endpoint.

Launching custom agents as OCI Hosted Applications requires containerizing the application logic and dependencies, pushing the compiled container image to OCI Registry (OCIR), and deploying it as a managed service that automatically configures storage, scaling, and public HTTPS gateways.

Explanation for Question 27:

CORRECT ANSWER: B

Correct Option details: Short-Term Memory (Conversations API session dialogue history), Long-Term Memory (cross-session user preferences and profiles), Context Compaction (automatic conversational summarization to fit token limits).

OCI Agents support three memory states: Short-Term session-based history managed natively via the Conversations API; Long-Term memory persisting structured user preferences and context across distinct sessions; and Context Compaction to summarize dialogue history and prevent token window overflows.

Explanation for Question 28:

CORRECT ANSWER: B

Correct Option details: The client and the MCP server communicate locally over standard input/output (stdio) pipes on the host machine with zero network traffic; only the isolated MCP server process initiates outbound HTTPS calls to the OCI Cloud API.

Under a local stdio MCP setup, the client application communicates with the local MCP server process entirely over stdin/stdout pipes, which incurs zero network overhead or sockets. Only the isolated MCP server process maintains outbound network sockets to communicate over HTTPS with the cloud backend API.

Explanation for Question 29:

CORRECT ANSWER: B

Correct Option details: To run model-generated Python code inside an isolated, secure compute sandbox with strict CPU, memory, and runtime limits to prevent host system compromise.

The managed Code Interpreter Tool acts as a secure container sandbox. It isolates execution of dynamically generated Python scripts (used for calculations and data filtering) under strict CPU, memory, and timeout limits, ensuring bad code cannot compromise production resources or databases.

Explanation for Question 30:

CORRECT ANSWER: B

Correct Option details: By leveraging Natural Language to SQL (NL-to-SQL) capabilities to translate natural language user questions into secure SQL statements and query structured data residing in Oracle Autonomous Databases.

OCI Enterprise AI Agents natively connect structured databases to LLM reasoning via Natural Language to SQL (NL-to-SQL) engines, translating user questions dynamically into SQL statements to query databases (such as Oracle Autonomous Database) without custom query pipelines.